Learning fixed-wing control at the actuator level with the recipe of *Learning to Fly in Seconds*

Clément Dumeril · MEng capstone, Agile Robotics and Perception Lab, UC Berkeley · advisor: Giuseppe Loianno

UC Berkeley Engineering — ARPL Agile Robotics and Perception Lab

Question

Can a policy that moves an airplane's control surfaces and throttle directly, at 50 Hz and with no autopilot below, be trained in minutes with the chain that flies quadrotors in *Learning to Fly in Seconds* [1] (its simulator structure, update intervals and reward form, here with SAC), and follow a moving point to within a meter?

Three steps

1. PX4's advanced_plane simulation: verify the method **done** → **2. Lab's Eratix** model from specs **first result** → **3. Real airplane** identified from flight data **next**

Method: the L2F chain

Airframe model Gazebo's AdvancedLiftDrag coefficients of PX4's advanced_plane

↓

Simulator 6-DOF, actuator lag and latency, RK4 at 50 Hz, in the structure of L2F; C++ with RLtools [3]

↓

SAC asymmetric actor-critic (the critics also see angle of attack and sideslip), L2F update intervals, reward weights grown in stages

↓

Checkpoint the most precise one with ≥ 97 % of validation flights completed

↓

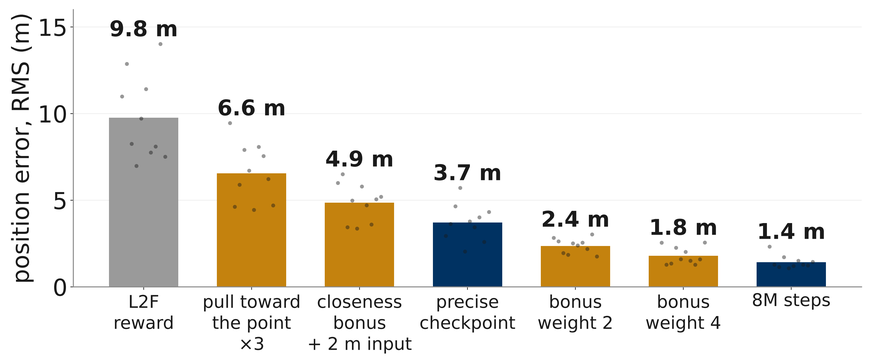
Tests 100 flights from a level start and 100 from random attitudes; points that turn

The C++ simulator matches the Python one to 7×10^{-13} over 7,136 steps. An early policy flies the airplane in PX4 SITL through ROS 2, where the simulator's one-step predictions have no bias against Gazebo (accelerations within 0.2 m/s^2 and 0.2 rad/s^2).

Task

A point starts up to 50 m away and flies straight at 12–18 m/s; the airplane starts level, or in a random attitude (bank $\pm 60°$, pitch $\pm 20°$), and must reach it and stay on it. Calm air, no sensor noise. The policy sees its position and velocity errors in the body frame and its last commands.

Step 1: precision, from 9.8 m to 1.4 m

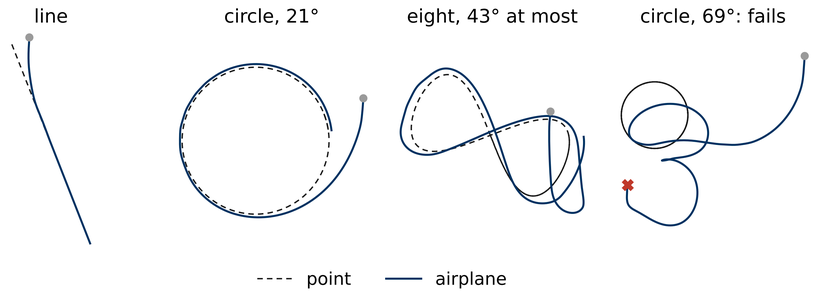


Test error over 10 training seeds (dots), best checkpoint, point flying straight; each bar keeps the changes on its left. Gold: changes of the reward (the third bar also adds the 2 m input).

$$r = 2 - \left[w_p \left(\frac{d}{20}\right)^2 + w_v \left(\frac{\|v - v^*\|}{4}\right)^2 + w_\beta \left(\frac{\beta}{5°}\right)^2 + w_{\Delta a} \|\Delta a\|^2 + w_a m_a^2 \right] + w_b \, e^{-(d/\sigma)^2}$$

$$v^* = v_{\text{ref}} + K \, (p_{\text{ref}} - p)$$

Squared costs are flat near the point: at the last stage, one meter of error costs 0.006 out of 2. Three changes give the policy a reason to close the last meters: a stronger pull of the velocity target toward the point (K: 0.1 to 0.3 s^{-1}), a bonus that pays only near the point, its width σ following half the mean training distance (weight 1, then 4), and the error also observed on a 2 m scale.

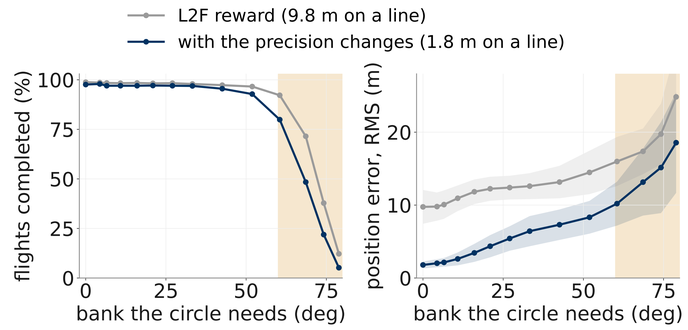


Policy at 1.8 m (4M steps), one test flight per path (dashed: the point, grey dot: the airplane's start). Trained only on straight lines.

1.4 m

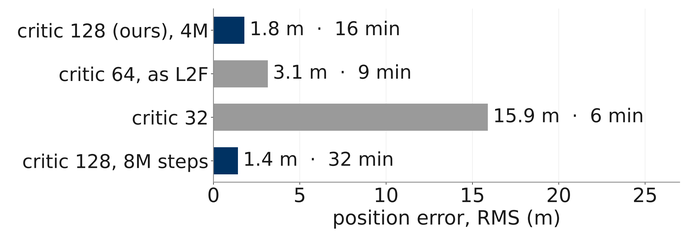
RMS position error in simulation, level starts, 10 seeds, 98.4 % of flights completed (96.7 % from random attitudes); 8M steps, 32 min on one CPU core. Left: a steady offset of about 1 m per axis.

Limit: points that turn



Up to 43° of bank the gain holds (7.3 m instead of 13.2 m), but the error grows as a lag; past 60° (shaded) the precise policies fail earlier. Next: train on turning points.

Training time



95 % of a training step is the network updates. Smaller critics lose the precision; more steps gain it.

Step 2: the lab's E-flite Eratix

99 % of test flights — **2.4 min** of training to fly — **7.9 m** error, slower point

83 g, modelled from its specs; point at 7–10 m/s, earlier reward. Next: the changes of step 1.

Step 3: the real airplane

Identify it from a few minutes of flight [2], retrain with wind, noise and randomized parameters, fly it through PX4. Not done yet.

[1] J. Eschmann, D. Albani, G. Loianno, *Learning to Fly in Seconds*, IEEE RA-L, 2024.
[2] J. Eschmann, D. Albani, G. Loianno, *Data-Driven System Identification of Quadrotors Subject to Motor Delays*, IROS, 2024.
[3] J. Eschmann, D. Albani, G. Loianno, *RLtools: A Fast, Portable Deep Reinforcement Learning Library for Continuous Control*, JMLR, 2024.

Code and results: github.com/clementdumeril/Fixed-Wing